

End-to-End Case Study

From closing tickets to fixing causes: a recurrence-based problem management analysis of 20,769 real ServiceNow incidents.

#	Section
1	Executive summary
2	The problem and the framework
3	Data and the incident-level model
4	Operational baseline
5	The DQ-08 bug, found and fixed
6	Recurrence detection
7	Choosing the candidate patterns
8	Investigation: fact to hypothesis
9	Intervention design and process change
10	Monitoring and the closed loop
11	Limitations and next steps
12	Reproducing this work

Prepared by Priyanka Karyampudi. Tools: MySQL 8, Python, Power BI. Companion documents: [01_Executive_Brief.pdf](#) and [03_Analytical_Appendix.pdf](#).

1. Executive summary

IT teams resolve incidents one at a time. What they rarely do is ask whether the same problem has happened before, who should own it, and whether a fix worked. This project takes a real ServiceNow incident log, cleans it into one row per incident (20,769 incidents), measures normal operational performance, detects recurring patterns, investigates the strongest two, and defines the metric that would prove each fix.

20,769

incidents

39.4%

SLA breach

73,359

recurrence pairs,
14 days

2

patterns
investigated

- **Baseline.** Category 34 is the outlier at about 58 days average resolution (439 incidents). Group 10 breaches SLA on 85.6% of incidents, with Groups 37 and 31 also well above the 39.4% overall rate; the largest group, Group 70, handles 7,369 incidents at 18.6%.
- **Recurrence.** The asset field is empty on 99.75% of incidents, so recurrence is measured by caller and category: 73,359 matching pairs within 14 days.
- **Pattern 1: Category 46** 1,993 incidents across 37 groups + unassigned, no clear owner. Hypothesis: a routing and ownership problem. Owner: Service Desk / IT Governance.
- **Pattern 2: Caller 1904 / Category 51.** 399 near-identical incidents, all handled by one group with no rework. Hypothesis: may be an automated source. Intervention: automation.
- **Monitoring.** Two KPIs with baselines: Category 46 SLA breach rate (54.1%), and Caller 1904 / Category 51 volume (399).
- **A bug found mid-analysis.** The final-row selection rule had an ordering bug (DQ-08) that materially understated SLA breach and overstated backlog. It was found, quantified, and every figure in this document was corrected — see section 5.

2. The problem and the framework

Where this project comes from

During my internship at Protiviti, while working on a client engagement, I noticed that incident management wasn't handled through any formal system. When something went wrong, it was simply communicated verbally: someone would flag it to the person responsible, and it would get resolved, informally, without being logged anywhere. It worked, in the sense that things got fixed.

But it made me pause and ask: what if a bigger incident came up, one that needed real investigation rather than a quick fix? And what if the same kind of issue kept happening, quietly, in the background, with no one able to see that pattern because nothing was ever recorded? There was no way to know. That question stayed with me, and it's what this project is built around: what becomes visible once incidents are actually captured, and what you can do with that visibility. The analysis itself runs on a separate, real, anonymised, publicly available ServiceNow incident dataset (not the client's own data), so that the question could be answered end to end without touching anything confidential.

Problem statement

Incident management at many organisations is reactive and case by case, with no structured way to tell whether an incident is part of a deeper, recurring problem. This project uses a real, anonymised enterprise incident dataset to demonstrate what becomes visible once incidents are properly recorded and analysed.

Stage	Question	Where in this project
Detect	Which incidents repeat?	Recurrence rule, sql/04, python/
Prioritise	Which patterns matter most?	Volume, SLA impact, reassignment, reopen (section 7)
Investigate	Why might it repeat?	Fact, pattern, hypothesis chain (section 8)
Decide	What kind of fix, and whose?	Intervention type and owner (section 9)
Act	What changes in the process?	AS-IS to TO-BE (section 9)

Stage	Question	Where in this project
Monitor	How will we know it worked?	KPIs with baselines (section 10)

3. Data and the incident-level model

Source: an anonymised event log from a ServiceNow instance used by an IT company, with 36 attributes per row (category, priority, assignment group, timestamps, SLA flag, reassignment and reopen counts, caller, asset).

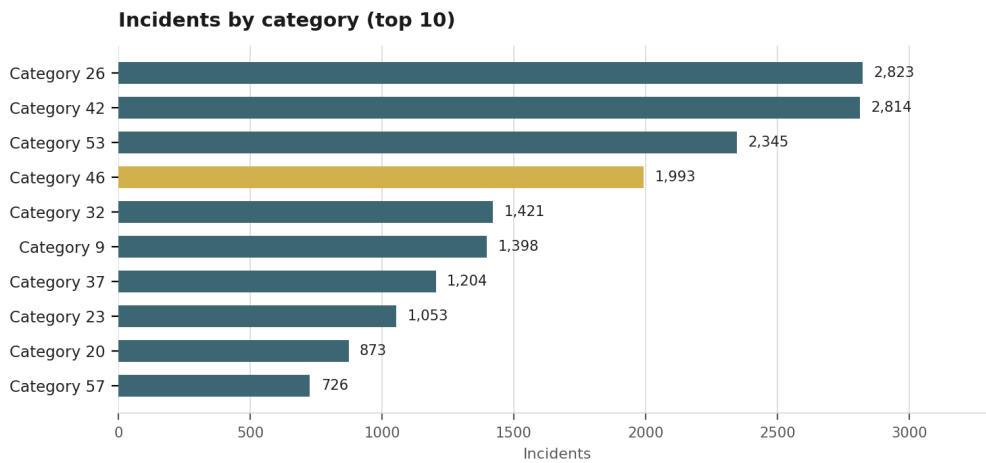
The first discovery: the file is not one row per incident. Every update writes a new row. The file has 119,998 rows but only 20,769 distinct incidents, about 5.8 rows each. One incident has 58 rows; the most common case is three rows (4,942 incidents).

Check	Result	Why it matters
Rows in the file	119,998	Event records, not incidents
Distinct incidents	20,769	The true unit of analysis
Most rows for one incident	58 (INC0019396)	Heavily reworked tickets exist
Terminal states	Resolved and Closed	Both are terminal (see section 5)
Date format	Day-first text, e.g. 29/2/2016 11:29	Sorting as text breaks chronology (DQ-08)
Missing-value marker	Literal ?	Asset missing on 20,717 incidents (99.75%)

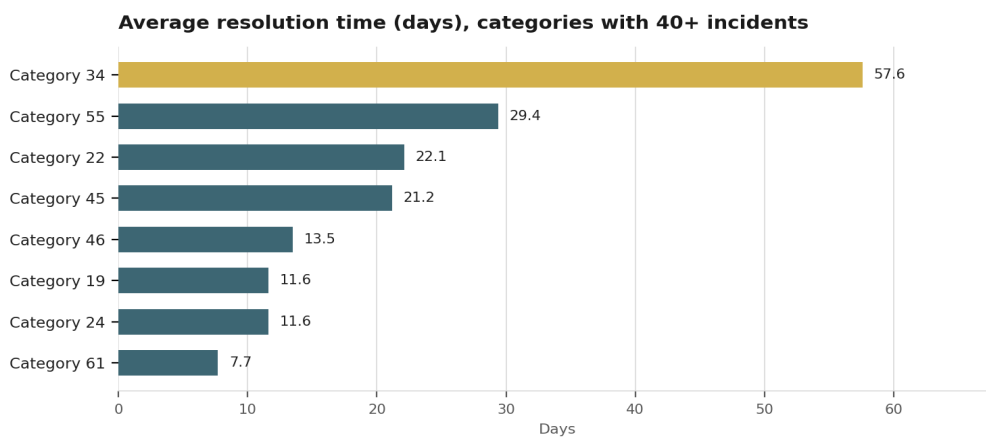
The fix. A window function keeps, for each incident, the most recently updated row (`ROW_NUMBER() OVER (PARTITION BY number ORDER BY sys_updated_at DESC, sys_mod_count DESC)`). The first version returned 22,136 rows instead of 20,769 because some incidents had two rows tied on the same timestamp; adding the modification count as a tie-breaker reconciled the view to exactly 20,769. Every later query reads from this view — though, as section 5 shows, getting the row count right did not mean the row-ordering itself was right.

4. Operational baseline

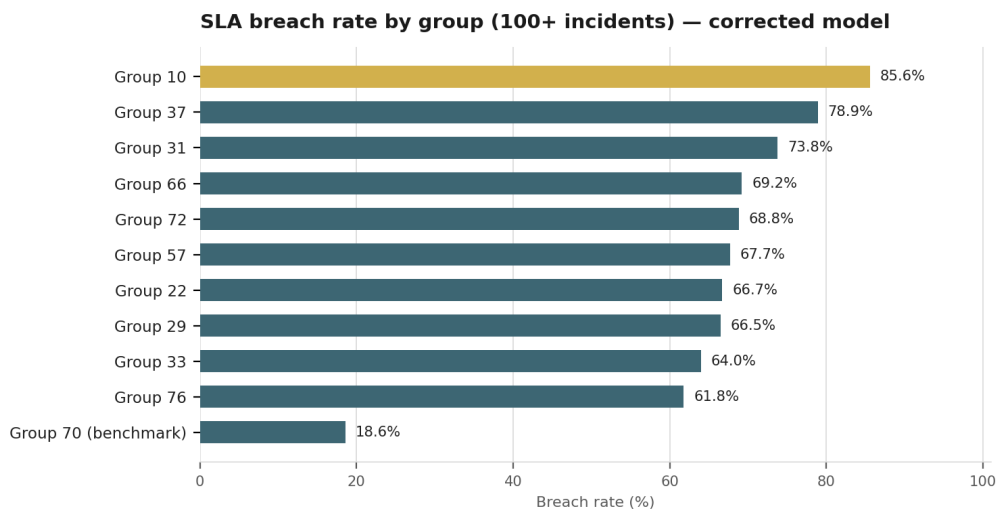
Before looking for problems, the baseline defines what normal looks like. Results are read with minimum sample sizes (groups with 100+ incidents, categories with 40+), and the ? placeholder is never treated as a real category or group.



Volume. Categories 26, 42, 53 and 46 carry the load. Volume alone is not a problem: Categories 26, 42 and 53 resolve well within normal ranges.



Resolution time Category 34 averages about 1,383 hours (57.6 days) across 439 incidents, roughly twice the next slowest meaningful category. Category 46 is high volume with a moderate 13.9-day average: worth checking against SLA and recurrence next.



SLA reliability (corrected model). Overall, 8,174 of 20,769 incidents breached SLA (39.4%). Group 10 leads at 85.6%, with Groups 37, 31, 66 and 72 also well above the overall rate. Group 70 shows that scale and reliability can coexist: 7,369 incidents at 18.6%. A further 2,157 incidents have no assignment group recorded (the ? group, 51.4% breach): a finding in itself, because unrecorded ownership is exactly the routing problem this framework looks for.

Backlog (corrected model) Just 1 record (INC0029233) still shows as active, but it is most likely not a real open ticket (it has resolved and closed dates). A major revision from the first figure; see section 5.

5. The DQ-08 bug, found and fixed

While double-checking a related date-parsing issue (two extreme callers whose activity span looked backwards until dates were parsed correctly — see section 7), the same class of bug was found in the incident-level model itself: the final-row rule ordered `sys_updated_at` as *text*, not as a date. Text sorting does not respect chronological order for day-first dates of varying digit width: '29/2/2016' sorts *after* '2/3/2016' alphabetically, even though it happened five days earlier.

This is not a hypothetical edge case. Checked directly against the raw log: for 8,018 of 20,769 incidents (38.6%), the buggy rule picked an earlier row — usually state Resolved, sometimes Active — instead of the true final row, which is state Closed for 20,768 of 20,769 (the one exception is INC0029233). Fifteen affected incidents were sampled and checked by hand against the raw event log; in every case, the corrected rule landed on the later, correct row.

Metric	v1 (buggy)	v2 (corrected)
Overall SLA breach	16.9% (3,517)	39.4% (8,174)
Backlog (still open)	38.6% (8,019)	0.0% (1 record)
Category 46 breach	25.9%	54.1%
Category 46 avg. reassignments	0.83	0.92
Group 70 breach	8.4% (of 7,189)	18.6% (of 7,369)
Recurrence, 14 days	73,339 pairs	73,359 pairs (unchanged)

Recurrence was barely affected because it depends on `opened_at`, `caller_id` and `category`, which don't change across an incident's event history — only the row-selection-dependent fields moved. This correction is logged in full in [documentation/Data_Quality_Log.md](#) (DQ-08, DQ-11).

6. Recurrence detection

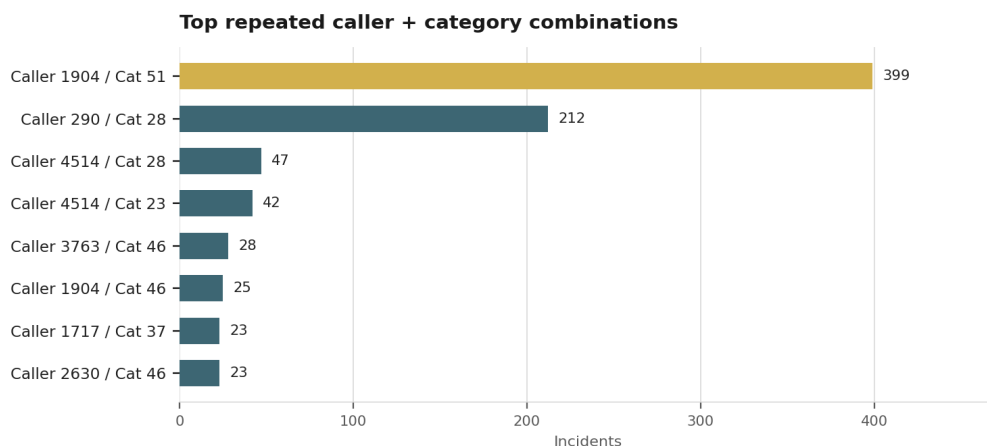
A slow or common category does not prove that a problem is recurring. Recurrence means the same thing breaking again, soon after the first fix.

Attempt 1: same asset, same category

The plan was to self-join the incident table on asset (cmdb_ci) and category and keep pairs opened within 14 days. Every row returned showed ? as the asset, meaning the query was matching any two incidents in a category. A coverage check confirmed why: only 52 of 20,769 incidents carry a real asset. The dataset cannot support asset-level recurrence. This is documented as a limitation rather than worked around.

Attempt 2: same caller, same category

The caller field is populated with no dominant placeholder. Re-running the join on caller and category returned 73,359 pairs within 14 days on the corrected model. That is too many to read: one caller with hundreds of incidents generates combinatorially many pairs. The redesign was to summarise instead of list, counting incidents per caller and category and keeping combinations seen three or more times.



Two things stand out. Caller 1904 raised 399 Category 51 incidents and Caller 290 raised 212 Category 28 incidents, numbers too extreme for a single person. And Category 46 appears repeatedly across at least fifteen different callers with 10 or more incidents each: a category recurring across many people rather than one anomalous account.

Sanity check and a second bug caught

To test whether the two extreme callers were a short automated burst, the first and last incident dates were queried. The first result showed Caller 290's first incident *after* its last one, because the dates were compared as text — the same bug class as DQ-08, caught independently in a different query. Parsing before aggregating gave the true spans: about two months each (Caller 1904 from 5 March to 9 May 2016; Caller 290 from 9 March to 12 May 2016). Sustained volume, not a one-day spike.

Sensitivity and robustness

Seven days may miss genuine repeats that take longer to resurface. Thirty days risks catching unrelated incidents that share a category. Fourteen days is a common operational cycle and a balance between the two — and this was tested, not assumed:

Window	Pairs (v2)	Incidents flagged	% of all incidents
7 days	51,014	4,213	20.3%
14 days	73,359	5,238	25.2%
30 days	100,701	6,406	30.8%

The pattern also survives excluding the two suspected automated callers: 7,291 / 9,647 / 13,322 pairs remain at 7, 14 and 30 days. The independent Python re-implementation (`python/recurrence_validation.py`) reconciled exactly to these SQL pair counts.

7. Choosing the candidate patterns

Candidates were ranked on frequency, SLA impact and rework (reassignment and reopen), then narrowed to two for depth. Depth on two patterns is more convincing than a shallow list of twenty.

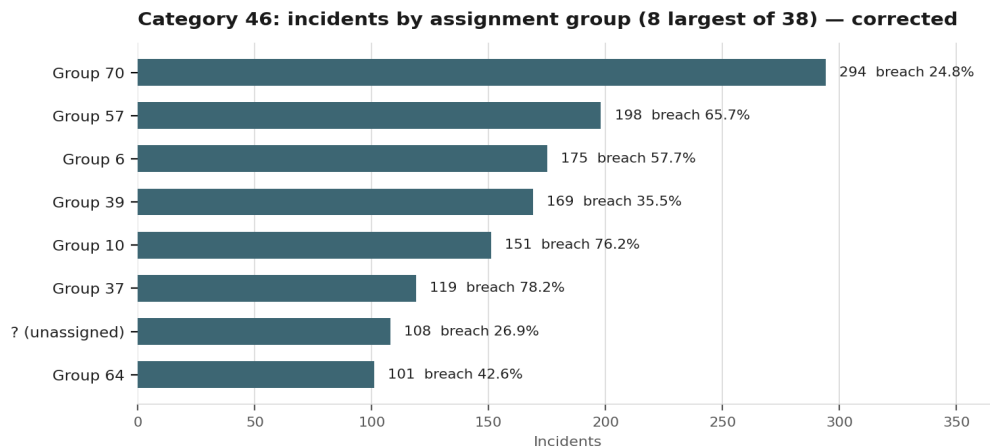
Candidate	Incidents	Why it ranks	Decision
Caller 1904 / Category 51	399	Most concentrated pattern in the data	Investigated (Pattern 2)
Caller 290 / Category 28	212	Second most concentrated; same shape	Logged as next candidate
Category 46 across callers	1,993 in category; 15+ callers	Cleanest systemic story: many people, many groups	Investigated (Pattern 1)
Caller 4514 / Categories 28 and 23	47 and 42	Human-scale repeat callers	Logged as next candidate

The two chosen patterns lead to different causes and different interventions, which shows that the analysis is not forcing every pattern into one explanation.

8. Investigation: fact to hypothesis

Each pattern is written as a chain: observed fact, pattern, contributing factor, hypothesis, and what must be validated. The data can support a hypothesis. It cannot prove a cause, and the text says so.

Pattern 1: Category 46



Link	Content
Observed fact	1,993 incidents over 37 named groups plus unassigned. The largest, Group 70, handles 294 (14.8%).
Pattern	Reassignments (0.92) sit close to the dataset-wide average (0.99): not a differentiator, despite looking elevated against the original, pre-correction baseline. Reopen rates are near zero in almost every group: fixes hold once made. Breach rates vary widely across groups (24.8% to 78.2%). SLA breach overall (54.1%) is well above the 39.4% dataset rate: this is the evidence that actually holds up.
Contributing factor	No single group owns more than 15% of Category 46. Ownership is diffuse: 37 teams + unassigned.
Hypothesis	No clearly defined owning team: tickets are routed reactively to whichever group picks them up, not by a rule matched to the category. Because reopens are near zero, fixes hold once the right team has the ticket: a routing and ownership problem, not a technical skill gap.
Needs validation	What Category 46 actually represents, and whether a triage or routing rule could be introduced.

Pattern 2: Caller 1904 / Category 51

Link	Content
Observed fact	399 Category 51 incidents from Caller 1904 over about two months.
Pattern	All 399 went to one group (Group 64). Average reassignments 0.0000, average reopens 0.0000, 22 breached SLA. Complete uniformity in handling.
Contributing factor	Caller 1904 may be a system or service account; requires validation.
Hypothesis	If automated, the 399 incidents may be one underlying condition logged repeatedly. The opportunity is not to investigate a person but to assess automated resolution or alert-threshold tuning, since the same team already resolves it identically each time.
Needs validation	Whether Caller 1904 is a person or an automated source, and what triggers each ticket.

9. Intervention design and process change

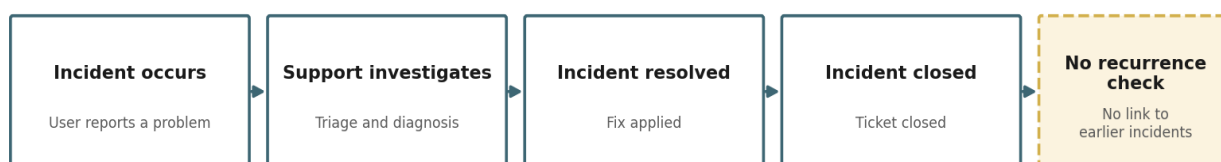
Each pattern is classified with one of five intervention types (technical fix, routing and ownership, knowledge gap, change-control weakness, automation opportunity) and given a proposed owner. These are hypotheses to validate with the service desk, not decisions.

	Pattern 1: Category 46	Pattern 2: Caller 1904 / Category 51
Intervention type	Routing and ownership problem	Automation opportunity
Proposed action	Define an owning team and a triage routing rule.	Assess automated resolution or alert-threshold tuning.
Proposed owner	Service Desk / IT Governance	Automation / Platform team with Group 64
Why this owner	The problem is upstream of any single technical team.	Group 64 already knows how to resolve it every time.

The recurrence rule. Flag an incident as a repeat when the same caller has raised the same category within 14 days of an earlier incident, and route it to root-cause review instead of closing it as a one-off.

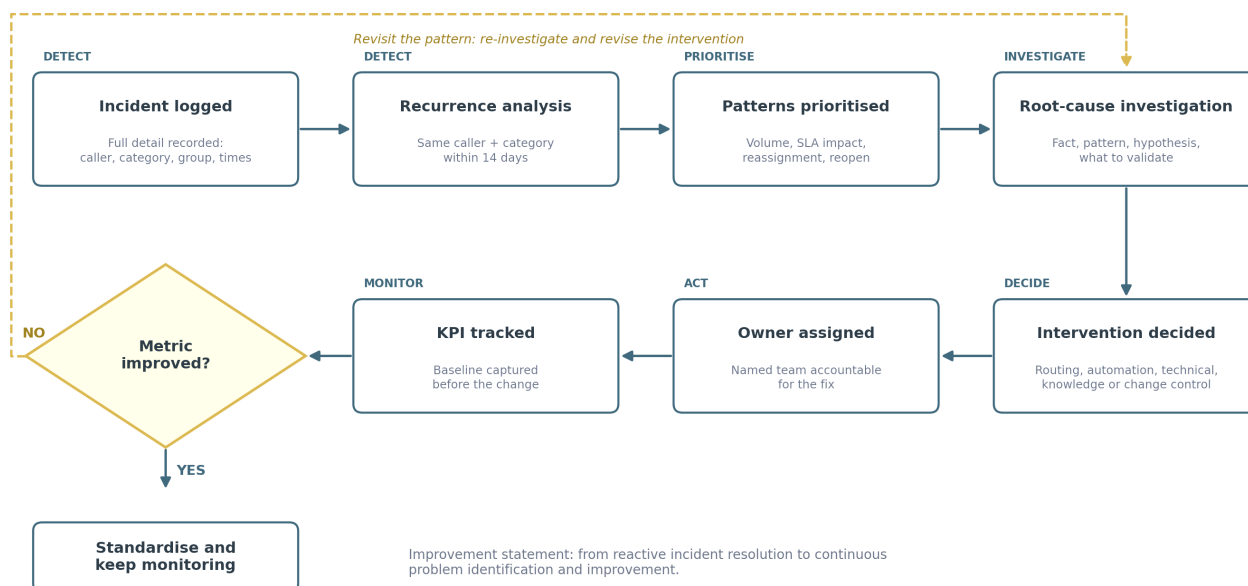
python/recurrence_validation.py applies the rule chronologically and writes the flagged incidents to CSV: the rule proven on the historical data, not deployed.

AS-IS: reactive incident handling



Each incident is fixed and closed on its own. Nothing asks whether it has happened before.

TO-BE: continuous problem identification and improvement



This moves the organisation from reactive incident resolution to continuous problem identification and improvement.

10. Monitoring and the closed loop

A recommendation without a metric cannot be tested. Each intervention has a baseline captured before any change, and a direction that would count as success.

Pattern	KPI	Baseline	Success looks like
Category 46	SLA breach rate	54.1%	Falls toward the dataset-wide 39.4%
Category 46	Average reassignment count	0.92 (in line with average)	Tracked, not expected to move on its own
Caller 1904 / Category 51	Incident volume	399	Drops sharply, ideally near zero, if automated

Closed loop. If these metrics do not improve after the intervention, the pattern is revisited: re-investigated, and the intervention revised. That is what makes this a loop instead of a one-time fix-and-forget recommendation.

11. Limitations and next steps

Limitation	Effect	Handling
Recurrence is by caller and category, not asset	Measures repeat reporting, not repeat failure of the same asset	Stated; asset field unusable (99.75% missing)
Caller 1904 and Caller 290 may be automated	Could inflate recurrence counts if excluded incorrectly	Kept and flagged; robustness run (sql/06 V3) shows the pattern holds without them
Dataset size differs from the source's public description	119,998 rows / 20,769 incidents loaded vs. 141,712 / 24,918 described publicly	Documented discrepancy, cause not confirmed (DQ-12); file spans 29 Feb - 13 May 2016; schema matches exactly
No real stakeholders	Owners and interventions are proposals, not decisions	Framed throughout as hypotheses requiring service-desk validation
Dashboard refresh needs local MySQL	The .pbix refresh needs a local MySQL database with incidents_final_v2	Values are documented in dashboard/README.md

Next steps

- Investigate Caller 290 / Category 28 and the Caller 4514 combinations with the same fact-pattern-hypothesis chain.
- Validate the open questions with a service desk owner: what Category 46 represents operationally, what Caller 1904 is, and why 2,157 incidents have no assignment group.

12. Reproducing this work

Step	File	Output
1	sql/01_data_profiling.sql	Load and profile: 119,998 rows, 20,769 incidents
2	sql/02_incident_level_model.sql	One row per incident; v1 superseded, v2 published
3	sql/03_operational_baseline.sql	Volume, resolution, SLA, backlog

Step	File	Output
4	sql/04_recurrence_detection.sql	Asset attempt, caller + category redesign
5	sql/05_candidate_investigation.sql	Evidence chains and KPI baselines
6	sql/06_sensitivity_validation.sql	7, 14, 30-day windows; robustness
7	python/recurrence_validation.py	Independent implementation and reconciliation
8	dashboard/	Three-page Power BI report